
EGB242 Assignment 2, Section 1 %%

Table of Contents

Initialise workspace	1
Section 1.1 plot audioMultiplexNoisy in frequency and time domain	1
Section 1.2 with function block	2
1.2 plot frequency 8210	3
1.2 plot frequency 24190	4
1.2 plot frequency 40210	5
1.2 plot frequency 56290	6
1.2 plot frequency 72160	7
Section 1.3	8
Section 1.4	9
Section 1.4 CF1	10
Section 1.4 CF2 2	11
Section 1.4 CF3	12
Section 1.4 CF4	13
Section 1.4 CF5	14
Section 1.5	15
helper function	15
1.5 CF1 clean audio signal	16
1.5 CF2 clean audio signal	16
1.5 CF3 clean audio signal	17
1.5 CF4 clean audio signal	17
1.5 CF5 clean audio signal	17

This file is a template for your MATLAB solution to Section 1.

Before starting to write code, generate your data with the ??? as described in the assignment task.

Initialise workspace

```
clear all; close all;
load DataA2 audioMultiplexNoisy fs sid;

% Begin writing your MATLAB solution below this line.R
load('DataA2.mat')
```

Section 1.1 plot audioMultiplexNoisy in frequency and time domain

```
%Create a time vector
samples = length(audioMultiplexNoisy);
timeVector = linspace(0, samples/fs, samples);

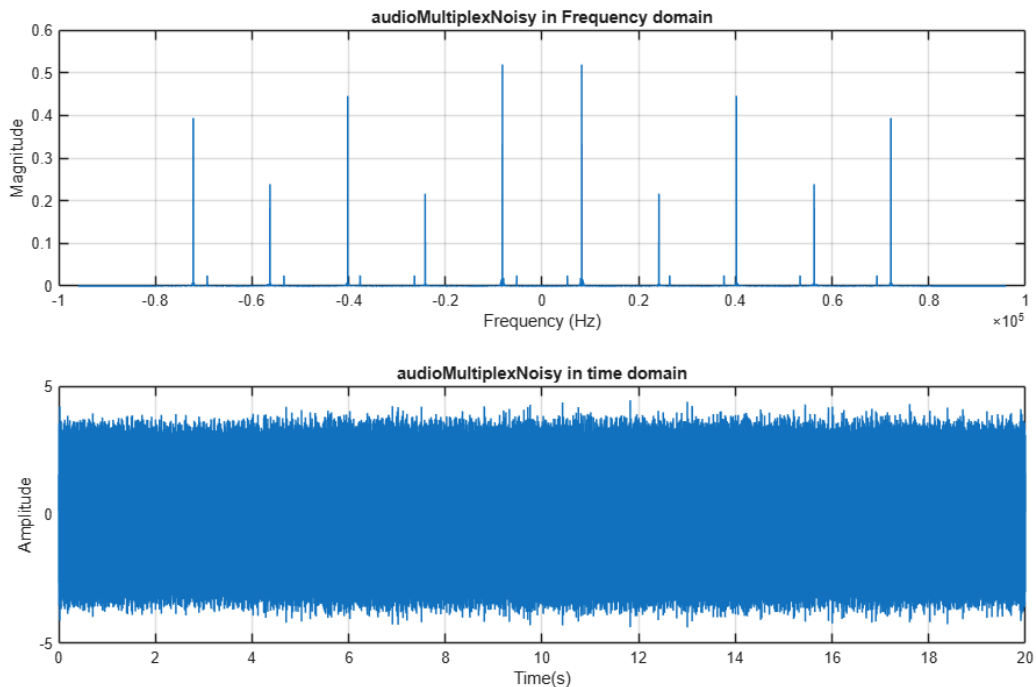
% Create a frequency vector
freq = linspace(-fs/2, fs/2, samples + 1);
freq(end) = [];
```

```
% Convert to the frequency domain
frequencySpectrum = fftshift(fft(audioMultiplexNoisy)) / samples;

% Plot the time domain vector
figure;
subplot(2,1,2);
plot(timeVector, audioMultiplexNoisy);
title('audioMultiplexNoisy in time domain');
xlabel('Time(s)');
ylabel('Amplitude');
grid on;

% plot in the frequency spectrum
subplot(2,1,1);
plot(freq, abs(frequencySpectrum));
title('audioMultiplexNoisy in Frequency domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
grid on;

% There are 5 carrier frequencies in the frequency domain. The plot above
% shows the frequency spikes at [8210, 24190, 40210, 56290, 72160]
```



Section 1.2 with function block

```
%Set the cutoff frequency
cutoffFreq = 4000;
```

```
% Using assignment 1 multiplexed system
demultiplex = fft(audioMultiplexNoisy) / fs;

% Define carrier frequency
CF1 = 8210;
CF2 = 24190;
CF3 = 40210;
CF4 = 56290;
CF5 = 72160;

% Create a function to modulate all frequencies using cos(2*pi*f*t);
function [c, demodulatedSignal, audioReceived, ...
        audioNormal, audioReceivedFreq] = ...
        carrierFreq(freq, timeVector, signal, cutoffFreq, fs)

    c = cos(2*pi*freq*timeVector);
    demodulatedSignal = signal.* c;
    audioReceived = lowpass(demodulatedSignal, cutoffFreq, fs);
    audioReceived = highpass(audioReceived, 2, fs);
    audioNormal = audioReceived/ max(abs(audioReceived));
    audioReceivedFreq = fft(audioReceived) / fs;

end

% Output the functions for each carrier frequency
[C1, demod1, audioreceived1, audioNormal1, audioReceivedFreq1] = ...
    carrierFreq(CF1, timeVector, audioMultiplexNoisy, cutoffFreq, fs);
[C2, demod2, audioreceived2, audioNormal2, audioReceivedFreq2] = ...
    carrierFreq(CF2, timeVector, audioMultiplexNoisy, cutoffFreq, fs);
[C3, demod3, audioreceived3, audioNormal3, audioReceivedFreq3] = ...
    carrierFreq(CF3, timeVector, audioMultiplexNoisy, cutoffFreq, fs);
[C4, demod4, audioreceived4, audioNormal4, audioReceivedFreq4] = ...
    carrierFreq(CF4, timeVector, audioMultiplexNoisy, cutoffFreq, fs);
[C5, demod5, audioreceived5, audioNormal5, audioReceivedFreq5] = ...
    carrierFreq(CF5, timeVector, audioMultiplexNoisy, cutoffFreq, fs);
```

1.2 plot frequency 8210

Listen to the sound of 8210 Hz

```
sound(audioreceived1, fs);

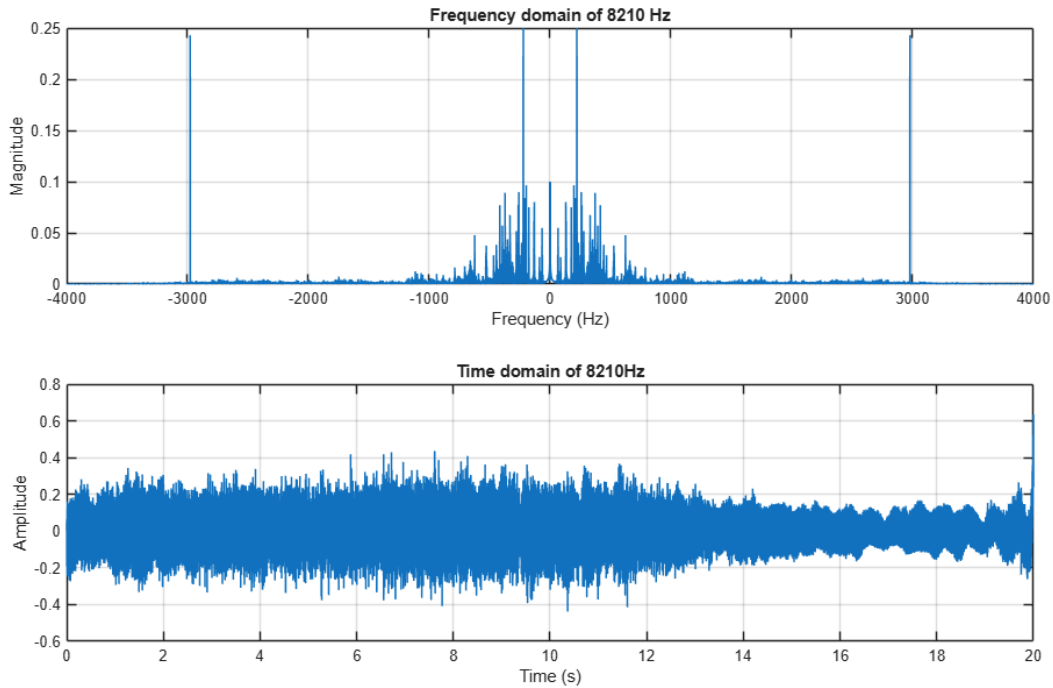
% Plot the time and Frequency domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(audioReceivedFreq1)));
title('Frequency domain of 8210 Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-4000, 4000]);
grid on;

subplot(2, 1, 2);
```

```

plot(timeVector, audioreceived1);
title('Time domain of 8210Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

```



1.2 plot frequency 24190

Listen to the sound of 24190 Hz

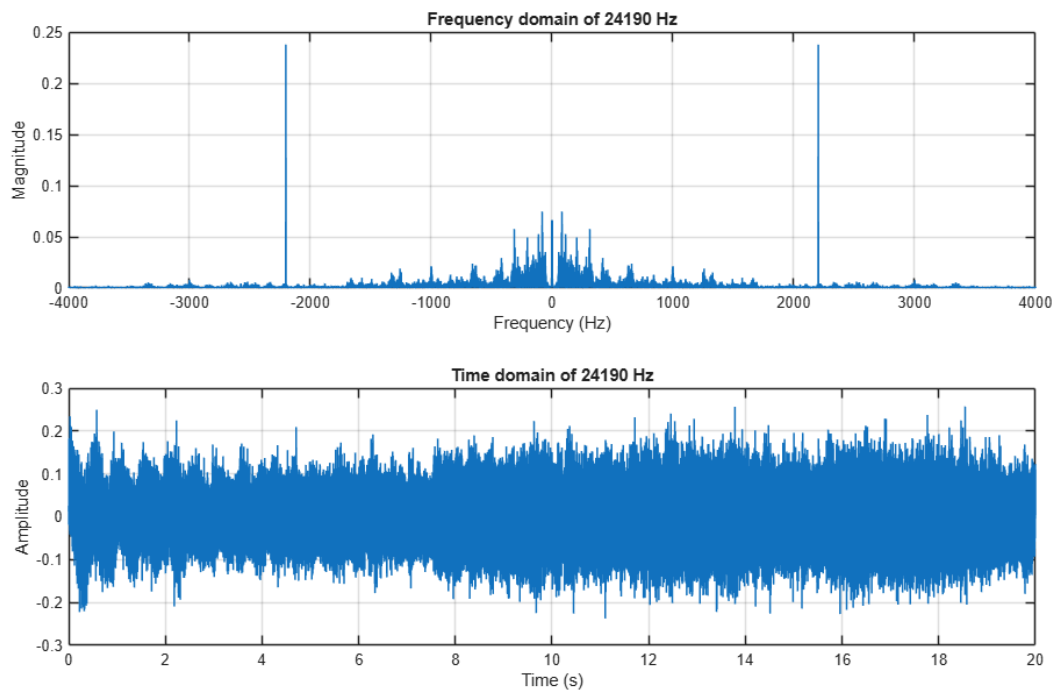
```

sound(audioreceived2, fs);

% Plot the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(audioReceivedFreq2)));
title('Frequency domain of 24190 Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-4000, 4000]);
grid on;

subplot(2, 1, 2);
plot(timeVector, audioreceived2);
title('Time domain of 24190 Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

```



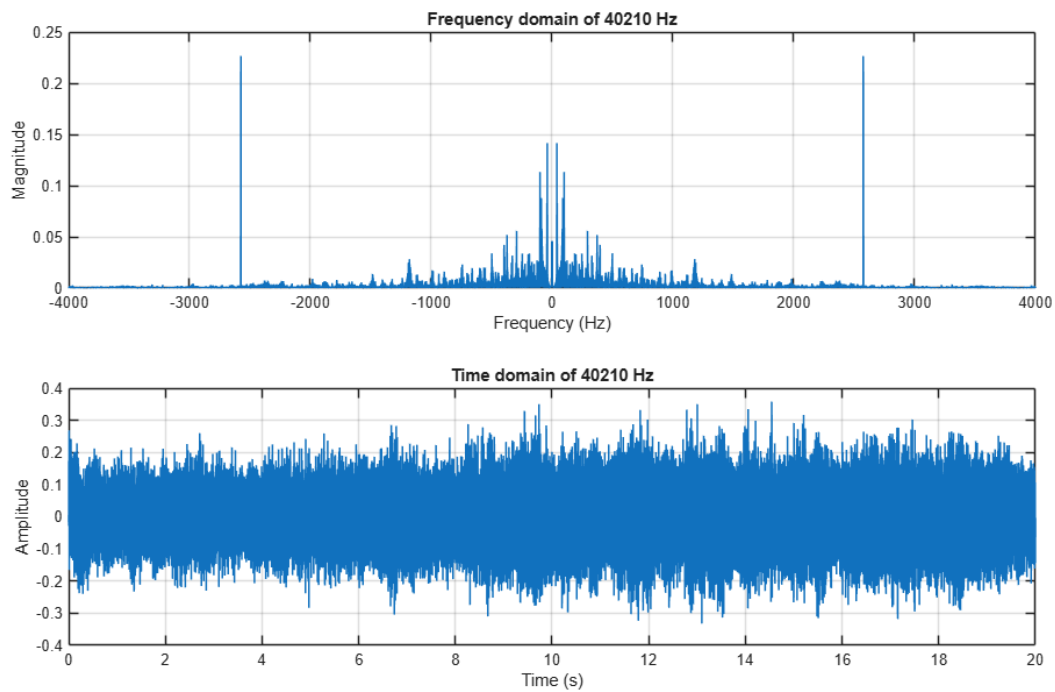
1.2 plot frequency 40210

Listen to the sound of 40210 Hz

```
sound(audioreceived3, fs);

% Plot the time and frequency domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(audioReceivedFreq3)));
title('Frequency domain of 40210 Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-4000, 4000]);
grid on;

subplot(2, 1, 2);
plot(timeVector, audioreceived3);
title('Time domain of 40210 Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



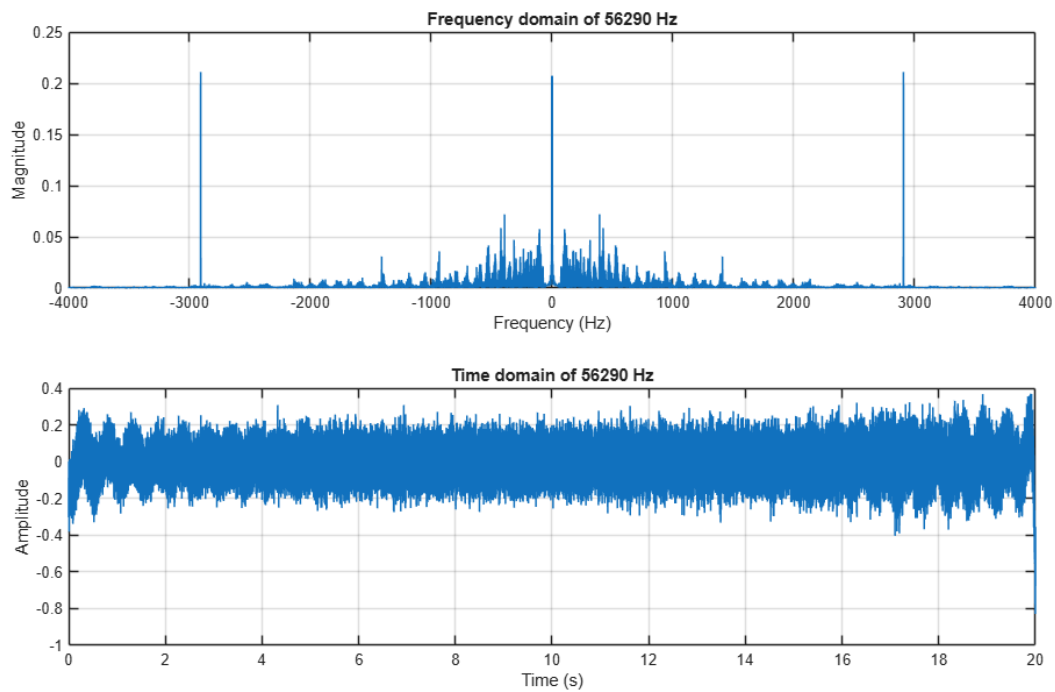
1.2 plot frequency 56290

Listen to the sound of 56290 Hz

```
sound(audioreceived4, fs);

%Plot the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(audioReceivedFreq4)));
title('Frequency domain of 56290 Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-4000, 4000]);
grid on;

subplot(2, 1, 2);
plot(timeVector, audioreceived4);
title('Time domain of 56290 Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



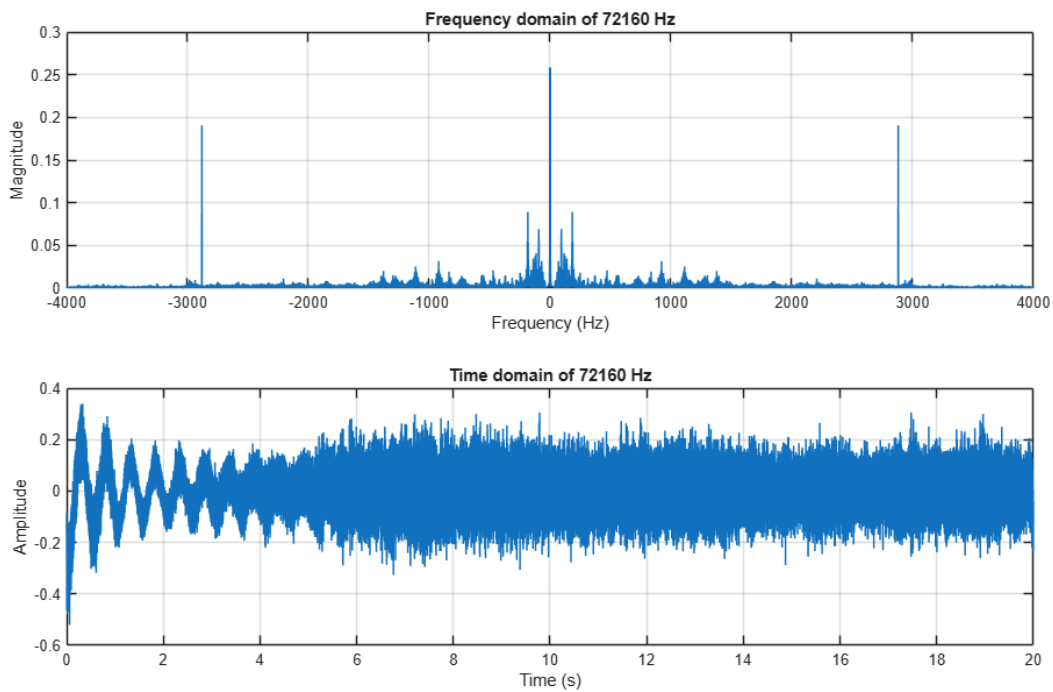
1.2 plot frequency 72160

Listen to the 72160 Hz

```
sound(audioreceived5, fs);

% Plot the time and frequency domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(audioReceivedFreq5)));
title('Frequency domain of 72160 Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-4000, 4000]);
grid on;

subplot(2, 1, 2);
plot(timeVector, audioreceived5);
title('Time domain of 72160 Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



Section 1.3

See tutorial 7 for a way to complete this assignment. Define time period

```
ts = 1/fs;

% Create an impulse
impulse = zeros(1, samples);
impulse(1) = 1/ts;

% Need to transfer the h function with the f(t)

%Use the channel feature to model the impulse response
channelOutput = channel(sid, impulse, fs);

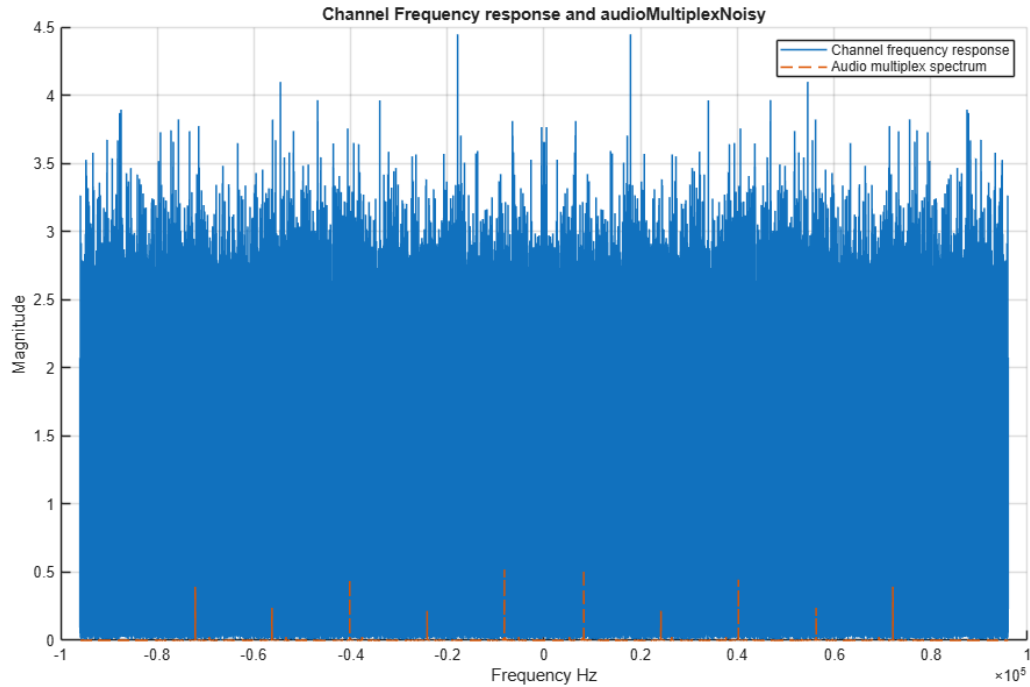
%Find the frequency response and create the frequency vector
channelFreq = fftshift(fft(channelOutput)) / fs;
channelFrequency = linspace(-fs/2, fs/2, length(channelOutput) + 1);
channelFrequency(end) = [];

%Plotting both the magnitude spectrum and multiplexed audio on the same
%graph
figure;
hold on;
plot(channelFrequency, abs(channelFreq));
plot(freq, abs(frequencySpectrum), '--');
title('Channel Frequency response and audioMultiplexNoisy');
xlabel('Frequency Hz');
ylabel('Magnitude');
```

```

legend('Channel frequency response', 'Audio multiplex spectrum');
%ylim([0, 0.05]);
%xlim([0, 0.05]);
grid on;
hold off;

```



Section 1.4

Reverse distortion for frequency 1

```

frequencyRev = fft(channelOutput, samples) / fs;

% Create the frequency domain vector
revfft = fft(audioMultiplexNoisy) / fs;

% Set the cut off frequency
cutoffFreq = 4000;

% Set the carrier frequency
CF1 = 8210;
CF2 = 24190;
CF3 = 40210;
CF4 = 56290;
CF5 = 72160;

% Create a function for reverse distortion
function [revAudio, revAudioEqualised, demRevSignal, cleanAudio,
CleanAudio]...
    = reverseDistortion(freq, cutoffFreq, fs, timeVector, frequencyRev,

```

```
revfft)

    % Reverse the channel distortion of the mulitplexed signal
    c = cos(2*pi*freq*timeVector);
    revAudio = revfft ./ frequencyRev;
    revAudioEqualised = ifft(revAudio) * fs;
    demRevSignal = revAudioEqualised .* c;
    cleanAudio = lowpass(demRevSignal, cutoffFreq, fs);
    cleanAudio = highpass(cleanAudio, 2, fs);
    CleanAudio = cleanAudio / max(abs(cleanAudio));

end

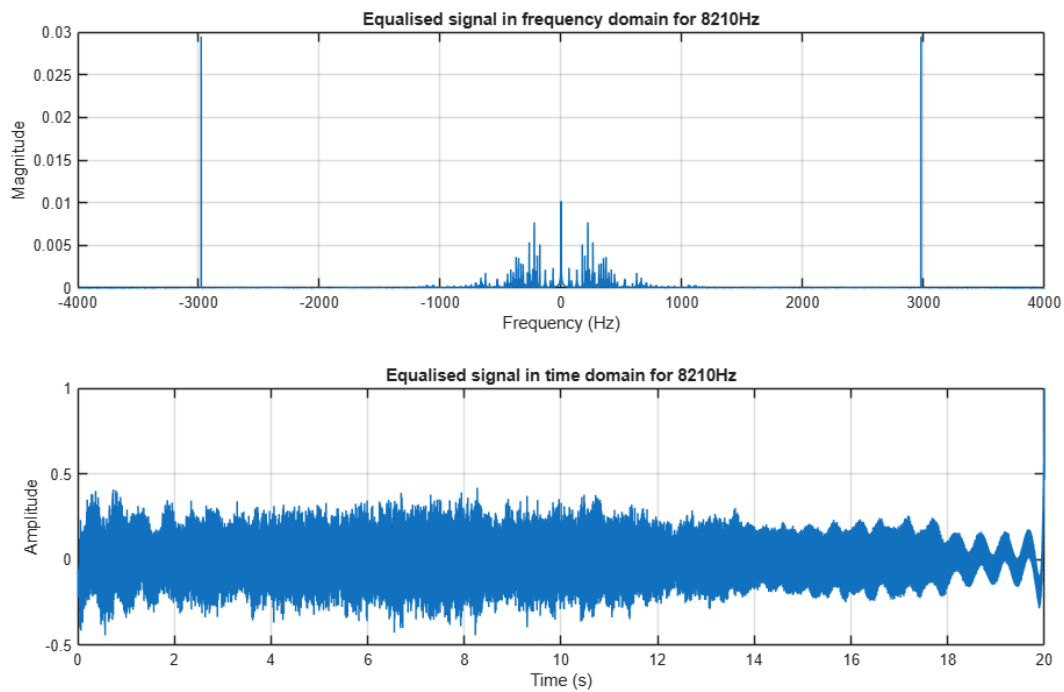
[revAudio1, revAudioEqualised1, demRevSignal1, cleanAudio1, CleanAudio1]...
    = reverseDistortion(CF1, cutoffFreq, fs, timeVector, frequencyRev,
revfft);
[revAudio2, revAudioEqualised2, demRevSignal2, cleanAudio2, CleanAudio2]...
    = reverseDistortion(CF2, cutoffFreq, fs, timeVector, frequencyRev,
revfft);
[revAudio3, revAudioEqualised3, demRevSignal3, cleanAudio3, CleanAudio3]...
    = reverseDistortion(CF3, cutoffFreq, fs, timeVector, frequencyRev,
revfft);
[revAudio4, revAudioEqualised4, demRevSignal4, cleanAudio4, CleanAudio4]...
    = reverseDistortion(CF4, cutoffFreq, fs, timeVector, frequencyRev,
revfft);
[revAudio5, revAudioEqualised5, demRevSignal5, cleanAudio5, CleanAudio5]...
    = reverseDistortion(CF5, cutoffFreq, fs, timeVector, frequencyRev,
revfft);
```

Section 1.4 CF1

```
%Listen to audio signal
sound(CleanAudio1, fs);

% Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(CleanAudio1) / samples)));
title('Equalised signal in frequency domain for 8210Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-cutoffFreq, cutoffFreq]);
grid on;

subplot(2, 1, 2);
plot(timeVector, CleanAudio1);
title('Equalised signal in time domain for 8210Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

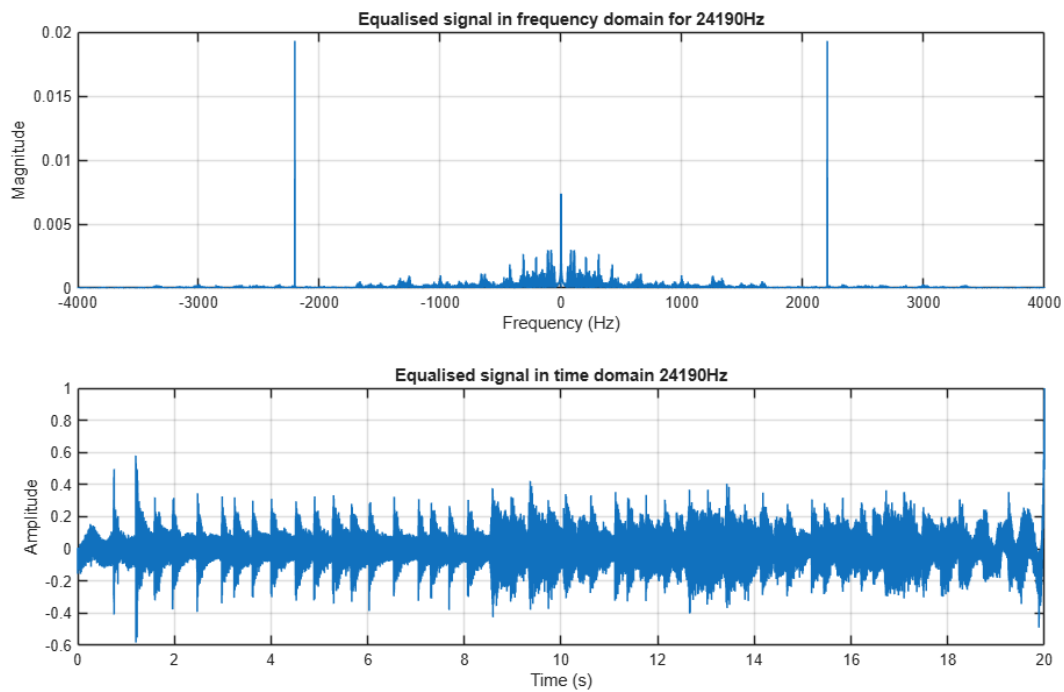


Section 1.4 CF2 2

```
%Listen to audio signal
sound(CleanAudio2, fs)

% Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(CleanAudio2) / samples)));
title('Equalised signal in frequency domain for 24190Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-cutoffFreq, cutoffFreq]);
grid on;

subplot(2, 1, 2);
plot(timeVector, CleanAudio2);
title('Equalised signal in time domain 24190Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

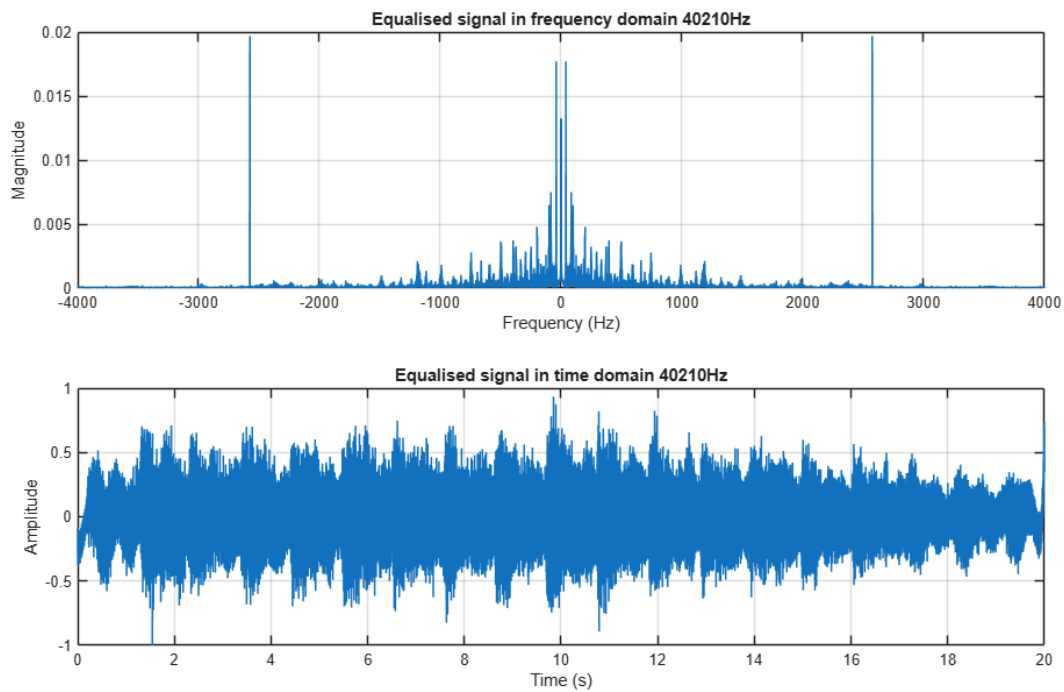


Section 1.4 CF3

```
%Listen to audio signal
sound(CleanAudio3, fs);

% Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(CleanAudio3) / samples)));
title('Equalised signal in frequency domain 40210Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-cutoffFreq, cutoffFreq]);
grid on;

subplot(2, 1, 2);
plot(timeVector, CleanAudio3);
title('Equalised signal in time domain 40210Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

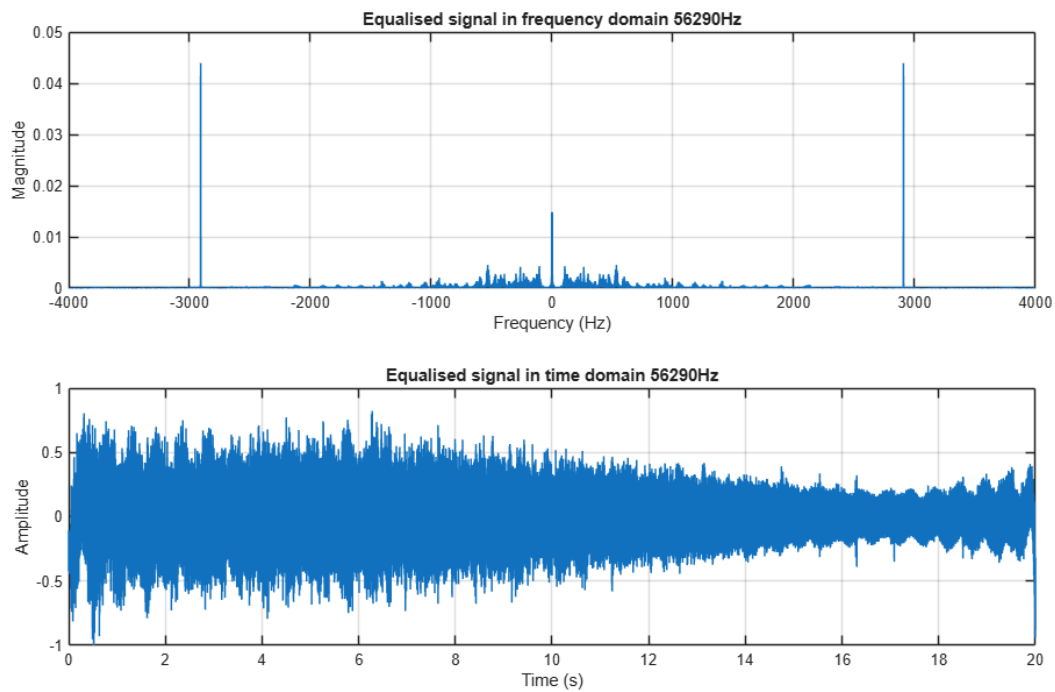


Section 1.4 CF4

```
%Listen to audio signal
sound(CleanAudio4, fs);

% Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(CleanAudio4) / samples)));
title('Equalised signal in frequency domain 56290Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-cutoffFreq, cutoffFreq]);
grid on;

subplot(2, 1, 2);
plot(timeVector, CleanAudio4);
title('Equalised signal in time domain 56290Hz');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

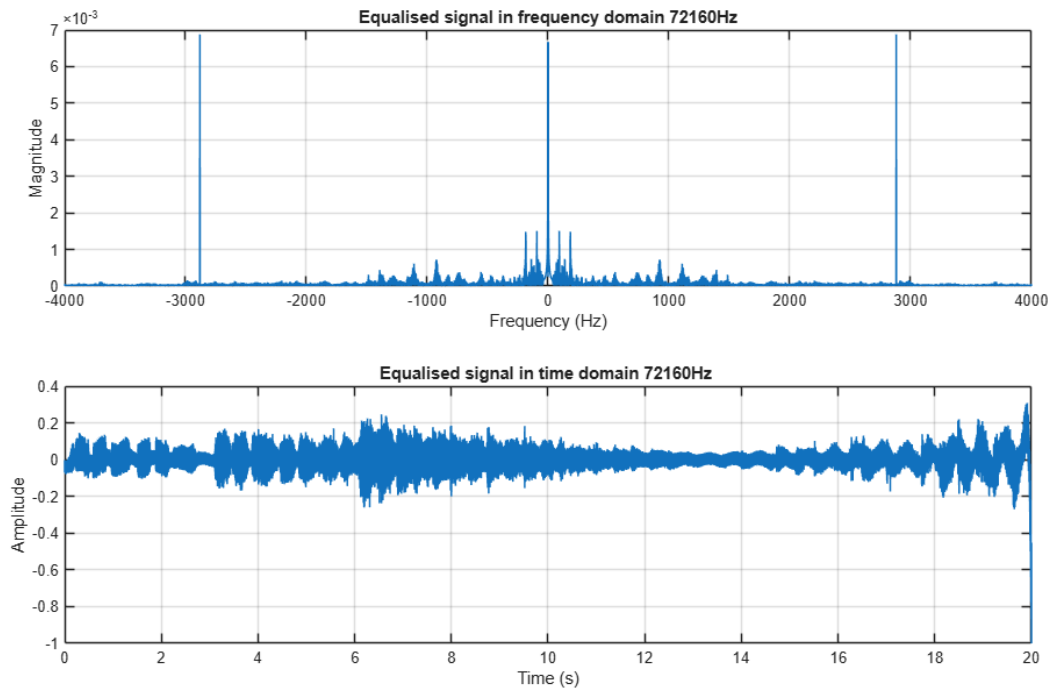


Section 1.4 CF5

```
%Listen to audio signal
sound(CleanAudio5, fs);

% Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(CleanAudio5) / samples)));
title('Equalised signal in frequency domain 72160Hz');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([-cutoffFreq, cutoffFreq]);
grid on;

subplot(2, 1, 2);
plot(timeVector, CleanAudio5);
title('Equalised signal in time domain 72160Hz ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```



Section 1.5

See tutorial 7 to complete the part. Estimate each channel's impulse and frequency response for equalisation

```
[h2, H2, f2] = impulseFreqResponse(@System2, impulse, fs);
[h3, H3, f3] = impulseFreqResponse(@System3, impulse, fs);
[h4, H4, f4] = impulseFreqResponse(@System4, impulse, fs);
[h5, H5, f5] = impulseFreqResponse(@System5, impulse, fs);
[h6, H6, f6] = impulseFreqResponse(@System6, impulse, fs);
```

```
% equalise and demodulate each carrier
channelResponses = {H1, H2, H3, H4, H5};
carrierFrequencies = [CF1, CF2, CF3, CF4, CF5];
finalAudio = cell(1, 5);
```

helper function

```
function [h, H, f] = impulseFreqResponse(system, imp, fs)
h = system(imp);
H = fft(h) / fs;
f = linspace(-fs/2, fs/2, length(h) + 1);
f(end) = [];
end
```

Undefined function 'System2' for input arguments of type 'double'.

```
Error in missionA2S1>impulseFreqResponse (line 383)
h = system(imp);
```

```
^^^^^^^^^^^
Error in missionA2S1 (line 370)
[h2, H2, f2] = impulseFreqResponse(@System2, impulse, fs);
^^^^^^^^^^^
```

1.5 CF1 clean audio signal

```
%Listen to the audio signal
sound(finalCleanAudio1,fs)

%Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(finalCleanAudio1)) / samples));
title('De-noised Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
% xlim([-cutoffFreq, cutoffFreq]);
grid on;

% Time domain
subplot(2, 1, 2);
plot(timeVector, finalCleanAudio1);
title('De-noised Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

1.5 CF2 clean audio signal

```
%Listen to the audio signal
sound(finalAudio2,fs)

%Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(finalAudio2)) / samples));
title('De-noised Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
% xlim([-cutoffFreq, cutoffFreq]);
grid on;

% Time domain
subplot(2, 1, 2);
plot(timeVector, finalAudio2);
title('De-noised Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

1.5 CF3 clean audio signal

```
%Listen to the audio signal
sound(finalAudio3,fs)

%Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(finalAudio3)) / samples));
title('De-noised Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
% xlim([-cutoffFreq, cutoffFreq]);
grid on;

% Time domain
subplot(2, 1, 2);
plot(timeVector, finalAudio3);
title('De-noised Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

1.5 CF4 clean audio signal

```
%Listen to the audio signal
sound(finalAudio4,fs)

%Plot in the frequency and time domain
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(finalAudio4)) / samples));
title('De-noised Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
% xlim([-cutoffFreq, cutoffFreq]);
grid on;

% Time domain
subplot(2, 1, 2);
plot(timeVector, finalAudio4);
title('De-noised Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

1.5 CF5 clean audio signal

```
%Listen to the audio signal
sound(finalAudio5,fs)

%Plot in the frequency and time domain
```

```
figure;
subplot(2, 1, 1);
plot(freq, abs(fftshift(fft(finalAudio5)) / samples));
title('De-noised Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
% xlim([-cutoffFreq, cutoffFreq]);
grid on;

% Time domain
subplot(2, 1, 2);
plot(timeVector, finalAudio5);
title('De-noised Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

Published with MATLAB® R2026a